

IBM — AWS DevOps Engineer

Interview Preparation Guide · 13 Questions & Answers

Q1 Self Introduction

Structure: Who → What → Why IBM

"I'm Sanket Chopade, a B.Tech Aeronautical Engineering graduate from RTM Nagpur University, 2025. Currently working as a DevOps Engineer Intern at Hisan Labs in Pune where I build and manage cloud-native infrastructure on AWS. My core stack includes AWS (EKS, EC2, S3, RDS), Terraform, Kubernetes, Docker, Jenkins, GitHub Actions, Prometheus, and Grafana. My flagship project, ApnaKart, is a 10-microservice e-commerce platform deployed on AWS EKS with full CI/CD pipelines and Datadog APM monitoring. I'm passionate about infrastructure automation and believe IBM's scale would give me the perfect environment to grow as a DevOps engineer."

■ *Tip: Keep it under 90 seconds. End with why IBM specifically — mention their hybrid cloud / Red Hat OpenShift work.*

Q2 Explain Kubernetes Architecture

Control Plane (Master Node):

Component	Role
API Server	Entry point for all kubectl commands and REST calls
etcd	Distributed key-value store — holds all cluster state
Scheduler	Assigns pods to nodes based on resource availability
Controller Manager	Runs controllers: ReplicaSet, Deployment, Node, etc.

Worker Nodes:

Component	Role
kubelet	Agent on each node — ensures containers are running
kube-proxy	Handles networking, maintains iptables rules
Container Runtime	Docker / containerd / CRI-O — runs actual containers

■ *Tip: In ApnaKart, the EKS control plane is managed by AWS. Mention that — it shows real-world awareness.*

Q3 Have You Worked on Auto-Scaling in Kubernetes?

Yes — in ApnaKart I configured HPA for microservices like order-service and product-service:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-service-hpa
spec:
```

```

scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: order-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
  resource:
    name: cpu
  target:
    type: Utilization
    averageUtilization: 70

```

Metrics were fed via **Prometheus + Metrics Server**. On AWS EKS, I also used **Cluster Autoscaler** to add/remove EC2 nodes based on pending pods.

■ *Tip: Mention both HPA (pod-level) and Cluster Autoscaler (node-level) — shows you understand both layers.*

Q4 Difference Between HPA and VPA

HPA — Horizontal Pod Autoscaler	VPA — Vertical Pod Autoscaler
• Scales by adding/removing pod replicas • Reacts to CPU, memory, or custom metrics • Best for stateless services with variable traffic • Does NOT change container resource limits • Works well with Cluster Autoscaler	• Scales by changing CPU/memory of existing pod • Requires pod restart to apply new resources • Best for stateful apps (databases, ML workloads) • Automatically sets resource requests & limits • Not ideal for rapid traffic spikes

■ *Tip: In ApnaKart I used HPA for microservices. VPA suits workloads where you can't scale horizontally.*

Q5 What Happens When a Pod Crashes?

#	Event	Detail
1	Container exits	Non-zero exit code triggers the crash detection
2	kubelet detects	Checks restartPolicy: Always / OnFailure / Never
3	Restart with backoff	Exponential backoff: 10s → 20s → 40s → max 5 min
4	CrashLoopBackOff	Pod status if it keeps failing repeatedly
5	ReplicaSet controller	Ensures desired replicas — schedules new pod on another node
6	Liveness probe	Can also trigger restart if container is running but unhealthy

Debug commands: **kubectl describe pod <pod>** and **kubectl logs <pod> --previous**

■ *Tip: Mention liveness vs readiness probes — shows you think proactively about health checks.*

Q6 How Do You Explore Applications Running on a K8s Cluster?

```
# List all pods across namespaces
kubectl get pods -A

# Describe a pod (events, mounts, resource usage)
kubectl describe pod <pod-name> -n <namespace>

# Live logs
kubectl logs <pod-name> -f

# Exec into a running container
kubectl exec -it <pod-name> -- /bin/bash

# Check services and endpoints
kubectl get svc,endpoints -n <namespace>

# Resource usage (requires metrics-server)
kubectl top pods
kubectl top nodes

# List all deployments
kubectl get deployments -A

# View ingress routes
kubectl get ingress -A
```

In ApnaKart, I also use **Datadog APM dashboards** and **Grafana** (fed by Prometheus) to explore service health visually.

■ *Tip: Mention Lens IDE or k9s as GUI tools — interviewers appreciate ecosystem awareness.*

Q7 Explain Terraform Modules in Your Project

Module	Responsibility
modules/vpc	VPC, subnets, IGW, NAT gateway, route tables
modules/eks	EKS cluster, node groups, IAM roles, OIDC provider
modules/rds	RDS PostgreSQL, subnet groups, parameter groups
modules/s3-cloudfront	S3 bucket + CloudFront distribution
modules/ecr	ECR repositories for all 10 microservices
modules/iam	IAM roles and policies with least-privilege access

Each module has **main.tf**, **variables.tf**, **outputs.tf**. Root module calls all of them with environment-specific tfvars files.

Remote state stored in **S3 with DynamoDB locking** for team safety.

■ *Tip: Always mention remote state with S3 + DynamoDB — it signals production maturity.*

Q8 Write a Sample Terraform Module to Launch an EC2 Instance

```

# modules/ec2/variables.tf
variable "ami_id" { type = string }
variable "instance_type" { type = string }
variable "subnet_id" { type = string }
variable "key_name" { type = string }
variable "instance_name" { type = string }

# modules/ec2/main.tf
resource "aws_instance" "this" {
  ami = var.ami_id
  instance_type = var.instance_type
  subnet_id = var.subnet_id
  key_name = var.key_name
  vpc_security_group_ids = [aws_security_group.this.id]
  tags = { Name = var.instance_name }
}

resource "aws_security_group" "this" {
  name = "${var.instance_name}-sg"
  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# modules/ec2/outputs.tf
output "instance_id" { value = aws_instance.this.id }
output "public_ip" { value = aws_instance.this.public_ip }

# root/main.tf — calling the module
module "web_server" {
  source = "../modules/ec2"
  ami_id = "ami-0abcdef1234567890"
  instance_type = "t3.micro"
  subnet_id = module.vpc.public_subnet_id
  key_name = "my-key"
  instance_name = "apnakart-web"
}

```

■ *Tip: Separate variables, main, and outputs cleanly — shows proper module design to the interviewer.*

Q9 Instance Types — Which Have You Used?

Family	Optimized For	Use Case
t3 / t3a	Burstable general purpose	Dev/test, bastion host
m5 / m6i	Balanced CPU + RAM	Production workloads, EKS nodes
c5 / c6i	Compute-optimized	CI/CD build servers

r5 / r6i	Memory-optimized	Databases, Redis, ML
g4dn	GPU instances	ML inference workloads
i3	Storage-optimized (NVMe SSD)	High I/O databases

Used in ApnaKart: t3.micro (bastion), t3.medium (EKS dev), m5.large (EKS prod/UAT), db.t3.medium (RDS PostgreSQL)

■ *Tip: Explain why: t3 for dev to save cost, m5 for prod for consistent CPU — shows cost awareness.*

Q1 0 Explain and Write Pipeline Stages in Your Project

```
// Jenkins Declarative Pipeline – ApnaKart
pipeline {
  agent { docker { image 'maven:3.9-openjdk-17' } }

  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Unit Test') {
      steps { sh 'mvn test' }
    }
    stage('Build') {
      steps { sh 'mvn package -DskipTests' }
    }
    stage('Code Quality – SonarQube') {
      steps { sh 'mvn sonar:sonar' }
    }
    stage('Docker Build & Push') {
      steps {
        sh 'docker build -t $ECR_REPO/$SERVICE:$BUILD_ID .'
        sh 'docker push $ECR_REPO/$SERVICE:$BUILD_ID'
      }
    }
    stage('Deploy to EKS') {
      steps {
        sh 'kubectl set image deployment/$SERVICE \
$SERVICE=$ECR_REPO/$SERVICE:$BUILD_ID -n production'
      }
    }
    stage('Smoke Test') {
      steps { sh 'curl -f http://$SERVICE_URL/actuator/health' }
    }
  }
  post {
    failure { slackSend message: "Build failed: ${env.JOB_NAME}" }
  }
}
```

■ Tip: Mention branch strategy: feature → dev → test → UAT → prod. Shows environment promotion maturity.

Q1 1 Write a Simple Dockerfile and Explain

```
# Stage 1: Build
FROM maven:3.9-openjdk-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline # cache deps layer
COPY src ./src
RUN mvn package -DskipTests

# Stage 2: Runtime (slim image)
FROM openjdk:17-jre-slim
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Multi-stage build: The builder stage compiles the app; the runtime stage copies only the final JAR. No Maven or JDK in production. Dependencies are cached in their own layer so rebuilds are fast when only source code changes.

■ Tip: This is already the answer to Q12 — multi-stage builds are the #1 way to reduce image size.

Q1 2 If Docker Image Size Is Huge, How Do You Reduce It?

Technique	Details
Multi-stage builds	Separate build tools from runtime image (shown in Q11)
Use slim/alpine bases	e.g. openjdk:17-jre-slim instead of ubuntu base
Minimize RUN layers	Chain commands with && and clean up in same layer
.dockerignore	Exclude node_modules, .git, test files, logs
No unnecessary packages	Use --no-install-recommends with apt-get
Distroless images	Google distroless: zero OS shell, minimal attack surface
dive / docker scout	Analyze each layer to find what's bloating the image

■ Tip: Real example: In ApnaKart I reduced image size from ~850MB to ~180MB using multi-stage + jre-slim base.

Q1 3 Difference Between ALB and NLB

ALB — Application Load Balancer	NLB — Network Load Balancer
• Layer 7 (HTTP / HTTPS / WebSocket) • Content-based routing: path, header, host • SSL/TLS termination at load balancer level • Best for microservices, APIs, and web apps • Supports AWS WAF integration • Used in ApnaKart for EKS ingress routing	• Layer 4 (TCP / UDP / TLS passthrough) • Routes based on IP and port only • Extreme throughput: millions of req/sec • Preserves original client source IP • Static IP / Elastic IP support • Best for gaming, gRPC, IoT, real-time

■ *Tip: In ApnaKart you use ALB via AWS Load Balancer Controller on EKS. NLB suits TCP passthrough or static IP needs.*